

prism — 个人量化交易系统

从数据采集到实盘执行的全栈自研 · 项目经历总结

项目类型	个人独立项目（自研 + AI 结对）
技术栈	Python / Flask / pandas / miniQMT / pytdx / SQLite
规模	272 次提交 · 28231 行生产代码 · 60 个测试文件
时间	2026-07-22 至今
产出	prism 选股引擎 · prism_web 控制台 · tt 做T策略 · 数据层与回测框架

一、项目概览

一句话：面向 A 股短线打板/趋势场景的自研量化系统：自建多源数据层、36 因子四层选股模型、防未来函数回测引擎、模拟盘/实盘执行链路，并配套两个可视化控制台，全部在一台 Windows 机器上闭环运行。

我的角色：独立完成需求定义、架构设计、编码、测试与运维；以「规格 → 计划 → 实现 → 双阶段评审 → 台账」的工程方法推进，AI 作为结对工程师参与实现与审查。

技术栈：Python 3.12 · Flask · pandas/numpy · miniQMT(xtquant) · 通达信 pytdx · 东财/新浪/FRED 开放接口 · SQLite · pytest(680+ 用例, 全离线确定性)

二、系统架构（七层，单机闭环）

层级	模块	职责与关键设计
数据层	prism/market_data · zt_history · tdx_source	多源降级链(QMT→通达信→东财→新浪/FRED)，增量续传+原子写，断源自愈，实测驱动的源可用性判定
因子层	prism/factors (36 个)	四层模型：市场门控 N / 首板 F / 妖股 Y / 势能板块 M·S·SEC；注册式扫描，因子体检覆盖合成场景
策略层	prism/engine · registry · strategies	JSON 策略文件 + 网页策略编辑器，指针热重载；三层等权打分，门禁与权重可调
回测层	prism/backtest · backtest/cli	严格 as-of 切片防未来函数；基本面缓存按日分区；OOS 样本外验证入口
执行层	prism/paper · paper_daemon · live_daemon · tt/	模拟盘账本(原子写、现金/持仓约束)；实盘信号守护；做 T 策略(底仓+网格+风控)接 miniQMT 直连
观察层	prism/sector_stage · sector_etf_map	板块感知：孕育期三信号、五阶段定位、资金惯性表；周度跟踪：周排名/ETF 锚点/60 日新高/纸面仓位上限定
界面层	prism_web · tt_web	Flask 双控制台(5000/5010)：选股、行情、因子库、策略编辑、回测、模拟盘、板块观察；只读监控台

三、技术壁垒与核心难点

1. 多源数据降级链与「断源自愈」

行情/资金/宏观三类数据分别来自 QMT、通达信、东财、新浪、FRED。每类都有可替换的备用源，失败时 fail-open 降级而不是让整条链路崩掉；源被封禁时按「实测」而非猜测切换（例如东财板块资金流历史端点长期封禁，实测确认同一主机的另一个端点可用后，改为当日快照前向累积，用时间换数据完整性）。

2. 防未来函数（回测可信度的底线）

回测按 as-of 日期对每个数据段做严格切片，任何晚于决策时点的数据不可见；基本面目缓存按 as-of 分日存储，窗口双向受限；对快照类因子在回测中直接剔除。这类问题不会被测试自动发现——必须有意识地把「未来信息」当成一类缺陷来审计。

3. 真实盘口排队成交模型

打板买入无法用「收盘价成交」近似：系统实现了买入排队状态机，成交条件是新增成交量穿越前方封单，并区分钉板（吃穿全队列）与炸板（吃穿初始队列）两条真实主通道；撤单流水带成交量归因。修正前九笔委托零成交——根因是原判定条件结构性不可满足。

4. 因子体系与「恒 0」根因定位

36

因子按四层加权组合；当因子实测恒为

0

时，系统先回答「是逻辑错还是数据没喂到」——因子体检用合成场景跑命中率，把恒

0

全部归因到数据供给断链（K 线根数差 1、字段语义冲突、指数 K 线未回填），而不是去改因子逻辑。

5. 板块感知层（从「捕捉信号」到「感知信号」）

用板块量价+成交额占比构建孕育期三信号（相对强度/占比抬升/结构改善）、五阶段定位（孕育-启动-主升-高潮-退潮）与资金惯性表（连续上榜=系统性增配），作为只读观察层先积累样本，验证有效后再考虑进入打分链路——把「改打分」与「改观察」严格分级，控制对真实交易的影响面。

6. 做 T 策略与真实账户对接

在既有选股链路之外独立实现日内

T+0：底仓+浮仓、网格档位、风控闸门、状态持久化；通过外部直连通道与 miniQMT 通信，dry-run 默认开启，真实报单需显式放行。

四、关键问题与解决过程（与 AI 协作的真实战役）

以下每个问题都真实发生过，且解决过程可复现、可验证：先定位根因，再选方案，最后用测试与实测数据确认修复。

git 对象库损坏 —— 从「仓库废掉」到完整恢复

现象：历史对象丢失、reflog 大量坏条目、fetch 协商卡死，远端交互彻底不通。处理：先用 fsck 定位损坏范围；对工作区仍存在的文件用 ``hash-object -w`` 重建同 SHA 的 blob 补回缺失对象；删除指向坏对象的引用；最后用 `clone --bare` 救援副本物理拷贝 pack/loose 对象进 `.git/objects`。结论：对象库损坏不必然丢历史，先定位、再按「本地文件哈希可无损重建」这一性质做抢救性恢复。

东财端点封禁 —— 用实测替代猜测的架构决策

外部建议引入「板块资金惯性表」，但历史资金流端点对 15/31 个行业持续封禁，多轮长间隔重试零成功。处理：不凭记忆下结论，写探针实测同一主机族的多个端点，发现「当日快照端点可用、历史端点不可用」，于是把设计从「回补历史」改为「每日快照前向累积」——牺牲即时完整性，换取长期可用。

数据「日期冻结」病 —— 两次同类缺陷的统一修复

现象：涨停池缓存停在 8-31，板块 K 线停在 9-02，重跑采集命令也不更新。根因：增量逻辑写成「已有缓存的标的直接跳过」，导致存量数据的尾部永远不会刷新（只有新标的会被采集）。处理：改为「尾部过期即重采」，并加「重采结果变短但日期未回退」的截断守卫，防止上游异常时静默缩短历史。同一类缺陷在两条缓存链路上先后出现——修复时一并抽象成通用判据。

封单强度因子单位错 —— 一个 $\times 100$ 的隐蔽错误

现象：封单强度因子实盘数值明显偏小。处理：追查 xtdata 官方示例确认盘口字段单位是「手」而非「股」，修正换算；随即用回测对比确认影响面——回测链路无实时盘口，该因子在回测中恒 0，因此修正只影响实盘/模拟盘。结论：修数值类缺陷必须同时回答「影响哪些链路」和「回测结论是否还成立」。

外部 AI 建议的取舍方法论 —— 从「照单全收」到「三问门」

把外部 AI 提出的升级建议逐项过三道门：当前数据层能否办到？取数是否容易？对系统实测是否有效？据此给出保留/降级/去掉三种结论，并交回需求方拍板。一次 4 模块的建议最终是「保留 2、换源 1、去掉 1」；另一次 5 项建议里 2 项因「系统已有人工入口」被查重拦下。该流程已固化为可复用的技能文档，避免重复建设与无效工程。

交付过程中的中断恢复 —— 把「上下文」当作可持久化的资产

长任务因额度/会话中断是常态。做法：每次实现都留「计划文件 + 任务简报 + 实现报告 + 台账」，中断后从仓库状态（未提交改动、测试红绿、报告结论）定位断点并续做，而不是重头再来。曾两次在「代码已写完、测试已绿、未提交」处接管并补完验收与提交。

五、项目成果与现状

工程规模	272 次提交（自 2026-07-22 起），生产代码 167 个文件 / 28231 行，测试文件 60 个
因子与策略	36 个自研因子，3 套可切换策略配置
测试	全仓离线测试一条命令跑完（禁网络、禁行情客户端）：prism 471 + prism_web 51 + datasource 123 + tt 148 + qmt_sync 27 + 根级 17 = 837 用例全绿；另有交付冒烟自检 ops/smoke_check.py 逐关验证真实缓存、路由与运行期状态
数据覆盖	31 个申万一级行业日 K + 成交额、5 千余只个股→行业映射、涨停池历史、全球指数/美债/VIX、ETF 锚点行情
执行链路	模拟盘（账本原子写、排队成交模型、绩效追踪）+ 实盘信号守护（未上线，验收流程就绪）+ 做 T 策略（真实账户直连，dry-run 默认开）
方法论沉淀	自研技能文档：升级建议 triage、方案评审、精简审计；规格/计划/报告/台账四件套贯穿每个批次

六、总结

这个项目最大的价值不在「用了多少技术」，而在于把一套完整的工程方法跑通：需求先过可行性门、设计先写规格、实现走测试驱动、每个批次经独立评审、所有决策与教训落进文档与台账。它证明了我在「没有团队、没有预算」的条件下，依然能把一个想法推进到稳定运行、可交付、可维护的状态，并且清楚知道系统的边界在哪里、下一步该补什么。

附：仓库为私有 Git 仓库，可按需提供代码走查或现场演示。